ASSIGNMENT ON LECTURE
DATABASE COURSE

RELATIONAL MODEL AND QUERY ON IT

Instructor:
Khabib Mustofa



submitted by
Ahmad Kailani
21/530323/SPA/01010

UNDERGRADUATE PROGRAM IN COMPUTER SCIENCE
DEPARTMENT OF COMPUTER SCIENCES AND ELECTRONICS
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA

2026

CONTENTS

Cover Page **i**

I Guidelines and Assignment Format **1**

1.1 Assignment Guidelines 1

1.2 Assignment Content Details 2

1.2.1 Case Domain 2

1.2.2 Output/Outcome of the Assignment 3

1.3 Sample Problem and Its Solution 4

II Results and Discussion **7**

2.1 Entity Relationship Diagram 7

2.2 Schema of Tables in the Database 8

2.2.1 SQL DDL Commands 8

2.3 Business Questions and Solutions Using SQL 11

2.3.1 Question number 1 11

2.3.2 Question number 2 12

2.3.3 Question number 3 14

2.3.4 Question number 4 15

2.3.5 Question number 5 17

REFERENCES **19**

LIST OF TABLES

LIST OF FIGURES

2.1 ER Diagram of the Academic Database 8

CHAPTER I

Guidelines and Assignment Format

Academic Integrity

All work submitted must be the result of the student's own thinking and work. Copying code or reports from other sources without clear attribution is a violation of academic integrity and will be subject to action in accordance with university regulations.

1.1 Assignment Guidelines

The assignment concerns the Relational Model and Queries on the Relational Model:

- Relational Algebra
- *Structured Query Language (SQL)*

The assignment answers are packaged in one chapter (chapter 2), whose sections have been provided in this template. The content of that chapter may be expanded as needed, but not reduced.

Deliverables

The assignment consists of four parts

1. Translate business requirements and processes into an ER diagram (*Entity-Relationship Diagram*). The proposal must include an explanation of the ERD creation process
2. Design and propose a relational database schema that aligns with the proposed ERD
3. Given a query, propose a representation of the query in relational algebraic expressions.
4. Propose a solution to the problem in the form of an SQL queries. If there are more than one possible query (with dif-

ferent operations or clauses), alternative queries must also be provided

Translated with DeepL.com (free version)

Assignment Conditions

1. The work may be done in groups with a maximum of 4 members.
2. Each group member **must** contribute adequately, and master the material produced by their group.
3. The work must be presented *clearly, completely, and neatly, including the use of formal language that is easy to understand*
4. The work must come from the team members' own thinking, and be agreed upon by all members

1.2 Assignment Content Details

1.2.1 Case Domain

It is desired to manage the academic data of a university with the following constraints:

Process Constraints

- Every academic activity needs to be recorded transparently and preserve chronology/history
- The data storage design accommodates the following matters and processes:
  - A student is allowed to retake a course without any limit on the number of times
  - The course grade (if retaken) used for the transcript is the **grade from the last attempt**
  - Courses are either Compulsory or Elective.

- A course may have other courses as prerequisites
- A lecturer may teach a course in a semester in more than one class (parallel)
- Some courses may be offered in even or odd semesters
- A class for a course is taught entirely by one lecturer (not team teaching)
- The course grades and weights given are

Grade	Weight
A	4.00
A-	3.75
A/B	3.50
B+	3.25
B	3.00
B-	2.75

Grade	Weight
B/C	2.50
C+	2.25
C	2
C/D	2.5
D	1.00
D	0.00

- The formula for calculating GPA from n courses is:

$$GPA = \frac{\sum_{i=1}^{n} \text{credits}_i.\text{weight}_i}{\sum_{i=1}^{n} \text{credits}_i}$$

1.2.2 Output/Outcome of the Assignment

The following are some **minimal** output/outcome to be addressed.

Deliverables

1. An E-R diagram (ERD) that matches the points above! (starting from identification: entities, relations, cardinalities, etc.)
2. A complete schema of the database tables that matches the ERD created; for example in the form

   ```
   Student(nim, name, gender, studyProgram,...)
   Lecturer(nip, ...)
   ```

   And **also SQL DDL commands** for defining the tables above (complete with all *constraints*)!

3. DML commands for the questions below:
   - **(a) List of elective course names and the courses that are their prerequisites (only name and course code information is sufficient). Courses that do not have prerequisites should still be displayed!**
   - **(b) List of elective courses taken by Computer Science students of the '2024' cohort in the odd semester of academic year 2026/2027!**
   - **(c) List of favorite elective courses for Computer Science students, namely the three courses with the most enrollees in the last three years!**
   - **(d) List of students who have a Semester Grade Point Average for the odd semester of 2025/2026 of no less than 3.25! (understand the difference between Semester GPA and Cumulative GPA)**
   - **(e) List of students and the Cumulative GPA they have obtained to date! (courses that have been retaken only need to be counted once)**
4. Relational algebra expression for each given query!

For each query above, the steps for working on it need to be described:

1. identify the tables that need to be involved
2. determine the join criteria and join type: INNER JOIN, NATURAL JOIN, LEFT/RIGHT JOIN or OUTER JOIN
3. Valid SQL commands. If a VIEW is needed as a temporary table, please also write it out in order.
4. if there is more than one *equivalent* SQL query, it will be a bonus if you can also write that query!

1.3 Sample Problem and Its Solution

Suppose the following query is given:

> *Display the list of elective courses that have never been taken/attended by students*

then an example answer is as follows:

Query number 1

1. **Tables needed in the query:** COURSE (MK), COURSEENROLLMENT (PK)
2. if using the SET DIFFERENCE operation there is no join process, so there is no join criterion. As an alternative, LEFT JOIN can be used, with the criterion `Mk.courseId= Pk.courseId`
3. **The required query**

```
SELECT * FROM Course Mk WHERE (type='E') and courseId NOT
    IN
    (SELECT courseId FROM CourseEnrollment Pk)
```

Listing I.1: SET DIFFERENCE approach

4. Equivalent SQL alternatives

```
SELECT Mk.* FROM Course Mk LEFT JOIN CourseParticipant Pk
    ON (Mk.courseId=Pk.courseId)
    WHERE Mk.type='E' and Pk.courseId is NULL
```

Listing I.2: Using LEFT JOIN

```
SELECT * FROM Course Mk WHERE (type='E') and
    NOT EXISTS
    (SELECT courseId FROM CourseEnrollment Pk
        WHERE Pk.courseId=Mk.courseId)
```

Listing I.3: Using EXISTS

5. Proposed Relational algebra expression

$$(\pi_{\text{courseId}}(\sigma_{\text{type='E'}}(\text{Course})) - \pi_{\text{courseId}}(\text{Course})) \bowtie \text{Course}$$

$$DIFF1 \leftarrow \pi_{\text{courseId}}(\sigma_{\text{type='E'}}(\text{Course}))$$
$$DIFF2 \leftarrow (DIFF1 - \pi_{\text{courseId}}(\text{Course}))$$
$$DIFF2 \bowtie \text{Course}$$

CHAPTER II

Results and Discussion

Group Members

1. Nashwan Hafid Andayuri, 25/554944/PA/23251
2. Ali Zainal Abidin 25/558945/PA/23497:
3. Farrell Rheza Rayhandanu 25/554767/PA/23237:
4. Adriel Gerald Tomyjune Bakara 25/558778/PA/23489

2.1 Entity Relationship Diagram

The database uses six main tables: `students`, `professors`, `courses`, `prerequisites`, `AvailableClasses`, and `CourseGrades`. Students, professors, and courses are identified by their NIM, professor ID, and course ID.

`AvailableClasses` records which class a student joins, who teaches it, and when it is taken. The records are kept after each semester so the enrollment history is not lost. The `prerequisites` table connects a course to the courses that must be completed before it.

`CourseGrades` stores the credits and weight received for every course attempt. A student can retake a course in another semester or academic year, and the latest attempt is used when calculating the cumulative GPA. The foreign keys connect these records to the correct student, course, and professor.

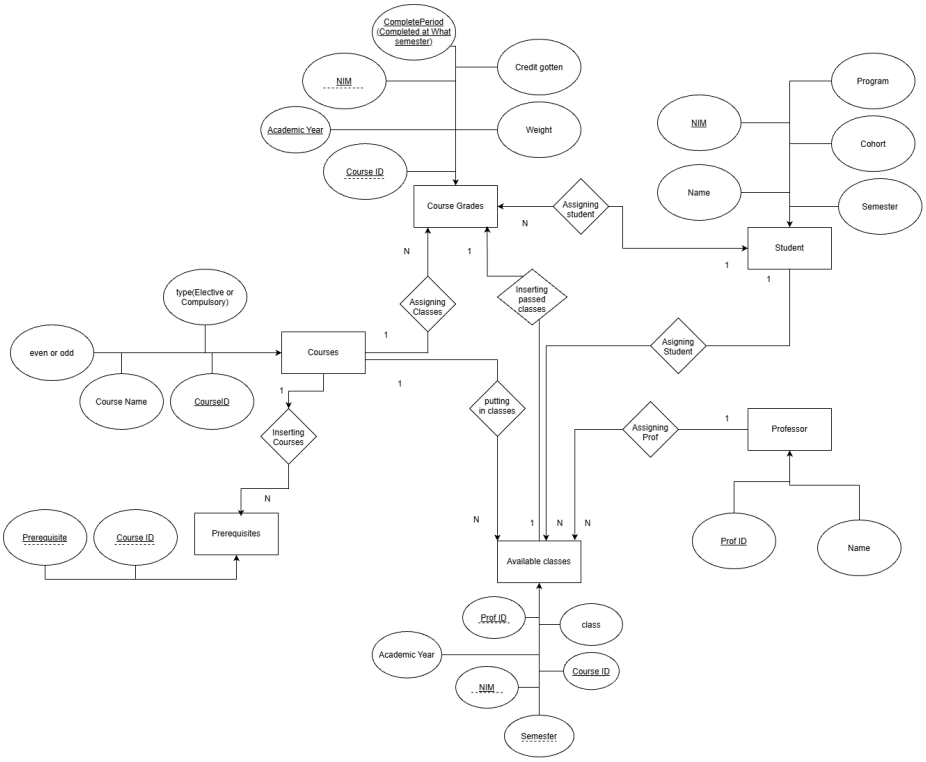


Figure 2.1: ER Diagram of the Academic Database

2.2 Schema of Tables in the Database

Table Schema in the Academic Database

- `students(nim, name, semester, cohort, program)`
- `professors(prof_id, name)`
- `courses(course_id, course_name, period, type)`
- `prerequisites(prerequisite, course_id)`
- `AvailableClasses(prof_id, nim, class, course_id, semester, academic_year)`
- `CourseGrades(course_id, nim, academic_year, completed_period, credits, weight)`

2.2.1 SQL DDL Commands

```
CREATE TABLE students (
    nim VARCHAR(20) PRIMARY KEY,
```

```
    name VARCHAR(100) NOT NULL,
    semester INT NOT NULL
        CHECK (semester > 0),
    cohort VARCHAR(4) NOT NULL
        CHECK (cohort ~ '^[0-9]{4}$'),
    program VARCHAR(100) NOT NULL
);

CREATE TABLE professors (
    prof_id VARCHAR(20) PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);

CREATE TABLE courses (
    course_id VARCHAR(20) PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL,
    period VARCHAR(10) NOT NULL
        CHECK (period IN ('Odd', 'Even', 'Both')),
    type VARCHAR(1) NOT NULL
        CHECK (type IN ('E', 'C'))
);

CREATE TABLE prerequisites (
    prerequisite VARCHAR(20) NOT NULL,
    course_id VARCHAR(20) NOT NULL,

    PRIMARY KEY (prerequisite, course_id),

    FOREIGN KEY (prerequisite)
        REFERENCES courses(course_id),

    FOREIGN KEY (course_id)
        REFERENCES courses(course_id),

    CHECK (prerequisite <> course_id)
);

CREATE TABLE AvailableClasses (
    prof_id VARCHAR(20) NOT NULL,
    nim VARCHAR(20) NOT NULL,
    class VARCHAR(20) NOT NULL,
    course_id VARCHAR(20) NOT NULL,
```

```
    semester VARCHAR(10) NOT NULL
        CHECK (semester IN ('Odd', 'Even')),
    academic_year VARCHAR(9) NOT NULL
        CHECK (academic_year ~ '^[0-9]{4}/[0-9]{4}$'),

    PRIMARY KEY (
        nim,
        course_id,
        semester,
        academic_year
    ),

    FOREIGN KEY (prof_id)
        REFERENCES professors(prof_id),

    FOREIGN KEY (nim)
        REFERENCES students(nim),

    FOREIGN KEY (course_id)
        REFERENCES courses(course_id)
);

CREATE TABLE CourseGrades (
    course_id VARCHAR(20) NOT NULL,
    nim VARCHAR(20) NOT NULL,
    academic_year VARCHAR(9) NOT NULL
        CHECK (academic_year ~ '^[0-9]{4}/[0-9]{4}$'),
    completed_period VARCHAR(10) NOT NULL
        CHECK (completed_period IN ('Odd', 'Even')),
    credits SMALLINT NOT NULL
        CHECK (credits > 0),
    weight DECIMAL(3,2) NOT NULL
        CHECK (weight BETWEEN 0.00 AND 4.00),

    PRIMARY KEY (
        course_id,
        nim,
        academic_year,
        completed_period
    ),

    FOREIGN KEY (course_id)
```

```
        REFERENCES courses(course_id),

    FOREIGN KEY (nim)
        REFERENCES students(nim),

    FOREIGN KEY (
        nim,
        course_id,
        completed_period,
        academic_year
    )
        REFERENCES AvailableClasses (
            nim,
            course_id,
            semester,
            academic_year
        )
);
```

2.3 Business Questions and Solutions Using SQL

2.3.1 Question number 1

List of elective course names and the courses that are their prerequisites (only name and course code information is sufficient). Courses that do not have prerequisites should still be displayed!

1. Tables needed to answer the query:
    - `courses` (aliased as `c`): stores the course code, course name, and type.
    - `prerequisites` (aliased as `p`): stores the prerequisite course code for each course.
    - `courses` (aliased as `pre_c`): used again to get the name of the prerequisite course.
2. Join criteria (if any):
    - `c.course_id = p.course_id`

- `p.prerequisite = pre_c.course_id`

A `LEFT JOIN` is used so elective courses without prerequisites are still shown.

3. SQL command according to the question:

```sql
SELECT
    c.course_id AS elective_course_code,
    c.course_name AS elective_course_name,
    p.prerequisite AS prerequisite_course_code,
    pre_c.course_name AS prerequisite_course_name
FROM courses c
LEFT JOIN prerequisites p
    ON c.course_id = p.course_id
LEFT JOIN courses pre_c
    ON p.prerequisite = pre_c.course_id
WHERE c.type = 'E'
ORDER BY c.course_id, p.prerequisite;
```

4. Relational Algebra:

$$\tau_{c.\text{course_id},p.\text{prerequisite}}\Big(\rho_{\text{renamed}}\Big(\pi_{\text{cols}}\Big((\sigma_{\text{type}='\text{E}'}(\rho_c(\text{courses})))\\ \bowtie_{c.\text{course_id}=p.\text{course_id}} \rho_p(\text{prerequisites})\\ \bowtie_{p.\text{prerequisite}=pre_c.\text{course_id}} \rho_{pre_c}(\text{courses})\Big)\Big)\Big)$$

5. Alternative SQL commands (if any) with adequate explanation: No alternative query.

2.3.2 Question number 2

> List of elective courses taken by Computer Science students of the '2024' cohort in the odd semester of academic year 2026/2027!

1. Tables needed to answer the query:

   - `students` (aliased as `s`): stores student information, including their program and cohort.

- `AvailableClasses` (aliased as `ac`): stores classes taken by students, including the semester and academic year.
- `courses` (aliased as `c`): stores the course code, course name, and type.

2. Join criteria (if any):
   - `s.nim = ac.nim`
   - `ac.course_id = c.course_id`
3. SQL command according to the question:

```sql
SELECT DISTINCT
    c.course_id,
    c.course_name
FROM students s
JOIN AvailableClasses ac
    ON s.nim = ac.nim
JOIN courses c
    ON ac.course_id = c.course_id
WHERE LOWER(s.program) = 'computer science'
    AND s.cohort = '2024'
    AND c.type = 'E'
    AND ac.academic_year = '2026/2027'
    AND ac.semester = 'Odd'
ORDER BY c.course_id;
```

4. Relational Algebra:

$$\tau_{c.\text{course_id}}\Big(\pi_{c.\text{course_id},c.\text{course_name}}\Big(\sigma_{\substack{\text{LOWER}(s.\text{program})=\text{'computer science'}\\ \wedge\, s.\text{cohort}=\text{'2024'}}}(\rho_s(\text{students}))\bowtie_{s.\text{nim}=ac.\text{nim}}\sigma_{\substack{ac.\text{academic_year}=\text{'2026/2027'}\\ \wedge\, ac.\text{semester}=\text{'Odd'}}}(\rho_{ac}(\text{AvailableClasses}))\bowtie_{ac.\text{course_id}=c.\text{course_id}}\sigma_{c.\text{type}=\text{'E'}}(\rho_c(\text{courses}))\Big)\Big)$$

5. Alternative SQL commands (if any) with adequate explanation: No alternative query. The semester is checked using `ac.semester` because it shows when the student took the course.

2.3.3 Question number 3

> List of favorite elective courses for Computer Science students, namely the three courses with the most enrollees in the last three years!

1. Tables needed to answer the query:
   - `courses` (aliased as `c`): stores the course code, course name, and type.
   - `students` (aliased as `s`): used to select Computer Science students.
   - `AvailableClasses` (aliased as `ac`): stores student enrollments and their academic years.
2. Join criteria (if any):
   - `ac.nim = s.nim`
   - `ac.course_id = c.course_id`
3. SQL command according to the question:

```sql
SELECT
    c.course_id,
    c.course_name,
    COUNT(DISTINCT ac.nim) AS total_enrollees
FROM AvailableClasses ac
JOIN students s
    ON ac.nim = s.nim
JOIN courses c
    ON ac.course_id = c.course_id
WHERE LOWER(s.program) = 'computer science'
    AND c.type = 'E'
    AND CAST(SUBSTRING(ac.academic_year, 1, 4) AS INT) >= (
        SELECT
```

```
            MAX(CAST(SUBSTRING(academic_year, 1, 4) AS INT)) -
                2
        FROM AvailableClasses
    )
GROUP BY c.course_id, c.course_name
ORDER BY total_enrollees DESC, c.course_id
LIMIT 3;
```

4. Relational Algebra:

$$\tau_{\text{total_enrollees}\downarrow,\, c.\text{course_id},\, \text{limit}=3}\Big(\gamma_{c.\text{course_id},c.\text{course_name}\,\text{COUNT(DISTINCT } ac.\text{nim)}\rightarrow\text{total_enrollees}}\Big(\sigma_{\substack{\text{LOWER}(s.\text{program})=\text{'computer science'}\\ \wedge\, c.\text{type}=\text{'E'}\\ \wedge\, \text{CAST(SUBSTRING}(ac.\text{academic_year},1,4)\text{ AS INT)}\\ \geq(\text{Subquery})}}\Big(\rho_{ac}(\text{AvailableClasses})\bowtie_{ac.\text{nim}=s.\text{nim}}\rho_s(\text{students})\bowtie_{ac.\text{course_id}=c.\text{course_id}}\rho_c(\text{courses})\Big)\Big)\Big)$$

5. Alternative SQL commands (if any) with adequate explanation: No alternative query. The subquery finds the latest academic year in the table, and subtracting two includes a total of three academic years.

2.3.4 Question number 4

> List of students who have a Semester Grade Point Average for the odd semester of 2025/2026 of no less than 3.25! (understand the difference between Semester GPA and Cumulative GPA)

1. Tables needed to answer the query:
   - `students` (aliased as `s`): stores the student's NIM and name.
   - `CourseGrades` (aliased as `cg`): stores the credits, grade weight, academic year, and completed period.

2. Join criteria (if any): `s.nim = cg.nim`
3. SQL command according to the question:

```sql
SELECT
    s.nim,
    s.name,
    ROUND(
        SUM(cg.credits * cg.weight) / SUM(cg.credits),
        2
    ) AS semester_gpa
FROM students s
JOIN CourseGrades cg
    ON s.nim = cg.nim
WHERE cg.academic_year = '2025/2026'
    AND cg.completed_period = 'Odd'
GROUP BY s.nim, s.name
HAVING SUM(cg.credits * cg.weight) / SUM(cg.credits) >= 3.25
ORDER BY semester_gpa DESC, s.nim;
```

4. Relational Algebra:

$$\tau_{\text{semester_gpa}\downarrow,\, s.\text{nim}}\Big(\sigma_{\text{semester_gpa}\geq 3.25}\Big(_{s.\text{nim},s.\text{name}}\gamma_{\text{ROUND}\left(\frac{\sum(cg.\text{credits}\cdot cg.\text{weight})}{\sum(cg.\text{credits})},2\right)\rightarrow\text{semester_gpa}}\Big(\rho_s(\text{students}) \bowtie_{s.\text{nim}=cg.\text{nim}} \sigma_{cg.\text{academic_year}=\text{'2025/2026'}\,\wedge\, cg.\text{completed_period}=\text{'Odd'}}(\rho_{cg}(\text{CourseGrades}))\Big)\Big)$$

5. Alternative SQL commands (if any) with adequate explanation: No alternative query. Only grades from the odd semester of 2025/2026 are used, so the result is a Semester GPA rather than a Cumulative GPA.

2.3.5 Question number 5

> List of students and the Cumulative GPA they have obtained to date! (courses that have been retaken only need to be counted once)

1. Tables needed to answer the query:
   - `students` (aliased as `s`): stores the student's NIM and name.
   - `CourseGrades` (aliased as `cg`): stores all course grades obtained by each student, including the academic year and semester.
   - `LatestCourseGrades`: selects the latest completed attempt when a student has taken the same course more than once.
2. Join criteria (if any): `s.nim = lcg.nim`. Only rows with `grade_rank = 1` are used so a repeated course is counted once.
3. SQL command according to the question:

```
WITH LatestCourseGrades AS (
    SELECT
        cg.nim,
        cg.course_id,
        cg.credits,
        cg.weight,
        ROW_NUMBER() OVER (
            PARTITION BY cg.nim, cg.course_id
            ORDER BY
                cg.academic_year DESC,
                CASE cg.completed_period
                    WHEN 'Even' THEN 2
                    WHEN 'Odd' THEN 1
                END DESC
        ) AS grade_rank
    FROM CourseGrades cg
    WHERE cg.weight IS NOT NULL
)
SELECT
    s.nim,
    s.name,
```

```
    ROUND(
        SUM(lcg.credits * lcg.weight) / SUM(lcg.credits),
        2
    ) AS cumulative_gpa
FROM students s
JOIN LatestCourseGrades lcg
    ON s.nim = lcg.nim
    AND lcg.grade_rank = 1
GROUP BY s.nim, s.name
ORDER BY s.nim;
```

4. Relational Algebra:

$$\tau_{s.\text{nim}}\Big(\gamma_{s.\text{nim},s.\text{name},\ \text{ROUND}\left(\frac{\sum(lcg.\text{credits}\cdot lcg.\text{weight})}{\sum(lcg.\text{credits})},2\right)\rightarrow\text{cumulative_gpa}}\Big(\rho_s(\text{students})\ \bowtie_{s.\text{nim}=lcg.\text{nim}}\ \rho_{lcg}\Big(\sigma_{\text{grade_rank}=1}\Big(\omega_{\substack{\text{grade_rank}\leftarrow\text{ROW_NUMBER}()\\ \text{PARTITION BY } cg.\text{nim},cg.\text{course_id}\\ \text{ORDER BY } cg.\text{academic_year}\downarrow,\ \text{completed_period}\downarrow}}\big(\sigma_{cg.\text{weight IS NOT NULL}}(\rho_{cg}(\text{CourseGrades}))\big)\Big)\Big)\Big)$$

5. Alternative SQL commands (if any) with adequate explanation: No alternative query. If a course was taken more than once, only the grade from the latest completed attempt is used. The academic year and semester are checked to find the latest attempt, so repeated courses are only counted once when calculating the Cumulative GPA.

REFERENCES